

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

**Институт информационных технологий и технологического образования
кафедра информационных технологий и электронного обучения**

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного
обеспечения и обработки больших данных»
форма обучения – очная

Курсовая работа

по дисциплине «Прикладные информационные технологии»
Разработка и контейнеризация веб-приложения для пересчета лабораторных
единиц анализа крови с использованием Docker и автоматизация выпуска
сертификатов Let's Encrypt.

Обучающейся 3 курса:
Чикиной Арины Олеговны

Руководитель:
ст. преп., Аксютин П. А.

Санкт-Петербург
2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. Теоретические основы разработки и контейнеризации веб-приложений.	5
1.1. Эволюция архитектур ПО. Монолитная архитектура как основа для малых веб-приложений	5
1.2. Технологии контейнеризации. Docker и его архитектура.....	7
1.3. Мультиконтейнерные приложения и Docker Compose. Сравнение с оркестраторами	8
1.4. Механизмы получения и автоматического обновления SSL/TLS-сертификатов Let's Encrypt ..	10
1.5. Обоснование выбора технологического стека и анализ альтернатив.....	12
ГЛАВА 2. Практическая реализация контейнеризации и автоматизации HTTPS.....	16
2.1. Структура проекта.....	16
2.2. Разработка Dockerfile для упаковки Flask-приложения	16
2.3. Конфигурация Docker Compose с поддержкой HTTPS.....	17
2.4. Принцип работы nginx-proxu и асме-companion	19
2.5. Запуск приложения на сервере и получение SSL-сертификата.....	20
ЗАКЛЮЧЕНИЕ	22
ЛИТЕРАТУРА	23
ПРИЛОЖЕНИЕ А	25
ПРИЛОЖЕНИЕ Б	26

ВВЕДЕНИЕ

Тема курсовой работы: разработка и контейнеризация веб-приложения для пересчета лабораторных единиц анализа крови с использованием Docker и автоматизация выпуска сертификатов Let's Encrypt.

Цель курсовой работы: разработка веб-приложения для пересчета лабораторных единиц анализа крови с использованием фреймворка Flask, контейнеризация приложения с помощью Docker, а также автоматизация выпуска и обновления SSL-сертификатов Let's Encrypt для обеспечения безопасного HTTPS-соединения.

Объектом исследования является веб-приложение для пересчета лабораторных единиц анализа крови, а также процессы его контейнеризации и развертывания с поддержкой HTTPS.

Данную цель лучше всего решать с помощью информационных технологий, так как это обеспечивает воспроизводимость окружения разработки и развертывания благодаря контейнеризации, автоматизацию получения и обновления SSL-сертификатов, а также удобный веб-интерфейс для конечного пользователя.

Задачи:

- Разработать веб-приложение на фреймворке Flask с HTML-формой для ввода данных и отображения результата,
- Создать Dockerfile для упаковки приложения в Docker-образ,
- Составить конфигурационный файл Docker Compose для локального запуска мультиконтейнерного приложения,
- Разработать production-конфигурацию Docker Compose с использованием nginx-проxy и acme-companion для автоматического получения и обновления SSL-сертификатов Let's Encrypt,

- Составить документацию по запуску проекта и разместить её в репозитории.

Структура работы:

- В главе 1 рассмотреть монолитную архитектуру веб-приложений как основу для разработки, технологии контейнеризации (Docker) и инструменты оркестрации (Docker Compose), а также механизмы получения и автоматического обновления SSL-сертификатов Let's Encrypt,
- Определить технологический стек (Flask, Docker, Docker Compose, nginx-проxy, аsme-companion) и провести сравнительный анализ альтернатив,
- В главе 2 реализовать веб-приложение на Flask с логикой пересчета лабораторных показателей и витаминов,
- Создать Dockerfile и конфигурационные файлы Docker Compose для локальной и production-среды, а также настроить автоматическое получение SSL-сертификатов Let's Encrypt.

ГЛАВА 1. Теоретические основы разработки и контейнеризации веб-приложений.

1.1. Эволюция архитектур ПО. Монолитная архитектура как основа для малых веб-приложений

Когда мы говорим о разработке программного обеспечения, архитектура показывает, как спроектировано решение с точки зрения обеспечения бизнес-логики, удобства использования и гибкости к дальнейшим изменениям [1, 23]. На ранних этапах разработки программного обеспечения начиналось с монолитной архитектуры. В настоящее время она остается одной из основных стратегий для создания небольших и средних информационных систем.

В программной инженерии монолитная модель относится к единой неделимой единице. Все компоненты — от серверного кода до пользовательского интерфейса — представляют собой единый объект, то есть объединены в одну программу, работающую на одной платформе [2, стр. 23]. В контексте данной курсовой работы монолитное приложение на Flask включает в себя как логику пересчета лабораторных единиц анализа крови, так и HTML-шаблоны для взаимодействия с пользователем, не требуя при этом отдельной базы данных или внешних сервисов.

Монолитная архитектура удобна для работы небольших групп и одиночных разработчиков. Её основными преимуществами являются [1, стр. 23]:

- Упрощенная разработка, тестирование и развертывание на старте — нет необходимости настраивать сетевое взаимодействие между несколькими сервисами.
- Единая кодовая база — весь проект хранится в одном репозитории, что упрощает навигацию и отладку.

- Высокая производительность — из-за отсутствия расходов на связь между сервисами (нет сетевых вызовов, всё работает через вызовы функций в общей памяти).

Эта архитектура является традиционным решением для создания приложений. Хотя некоторые разработчики считают её устаревшей, монолитная архитектура остается идеальным решением для стартапов и небольших проектов, которым необходимо как можно быстрее запустить продукт в эксплуатацию. В частности, для веб-приложения-калькулятора пересчета лабораторных показателей крови монолит является оптимальным выбором, так как:

Приложение не требует хранения пользовательских данных или истории запросов.

Все вычисления выполняются на стороне сервера мгновенно, без необходимости в распределенных транзакциях.

Логика пересчета (коэффициенты для глюкозы, холестерина, креатинина) статична и не требует частого обновления.

Однако у монолитной архитектуры есть и ряд ограничений, которые важно учитывать при проектировании [1, стр. 23]:

- Громоздкая кодовая база — с течением времени, если продукт продолжает развиваться, его структура может стать размытой, а код — трудно поддерживаемым.
- Сложность внедрения новых технологий — замена одного компонента (например, веб-сервера) может потребовать переписывания всего приложения.
- Недостаточная гибкость — в монолитных приложениях каждое небольшое обновление требует полного повторного развертывания контейнера или перезапуска сервера.

Несмотря на эти недостатки, для задач, решаемых в данной курсовой работе, монолитная архитектура является наиболее рациональным выбором. Применяемый фреймворк Flask изначально спроектирован для создания

легковесных монолитных веб-приложений, а Docker позволяет упаковать такой монолит в контейнер, сохраняя все преимущества изоляции окружения без необходимости дробления на микросервисы.

1.2. Технологии контейнеризации. Docker и его архитектура

При разработке приложений часто возникает проблема несовместимости окружений: код, работающий на компьютере одного разработчика, может не запуститься на другом из-за разных версий интерпретаторов, библиотек или операционной системы [3, 23]. Контейнеризация решает эту проблему.

Docker — платформа контейнеризации с открытым исходным кодом, позволяющая упаковать приложение со всеми зависимостями в изолированную среду — контейнер [1, стр. 23]. Контейнер можно запустить на любой системе с установленным Docker, и результат всегда будет одинаковым (свойство воспроизводимости).

Чтобы понять преимущества Docker, полезно сравнить контейнеризацию с виртуальными машинами (ВМ). Виртуальная машина эмулирует полноценный компьютер со своей гостевой операционной системой, драйверами и файловой системой. Гипервизор распределяет ресурсы хоста между ВМ. Такой подход надёжен, но тяжёл: каждая ВМ занимает гигабайты памяти, запускается минутами [4, стр. 23].

Docker использует контейнеризацию на уровне операционной системы. Контейнеры делят ядро хостовой системы, но имеют собственные изолированные пространства [1, стр. 23]:

- namespaces (пространства имён) — изолируют процессы, сеть, файловую систему, пользователей;
- cgroups (control groups) — ограничивают потребление ресурсов (CPU, RAM) каждым контейнером.

В результате контейнеры запускаются за секунды, занимают мегабайты и имеют минимальные накладные расходы [3, стр. 23].

Архитектура Docker построена по клиент-серверной модели [1, стр. 23]:

- Docker CLI — командная строка для управления Docker (docker build, docker run и др.).
- Docker Daemon (dockerd) — фоновый процесс, который непосредственно создаёт и управляет контейнерами.
- Docker Registry (реестр) — хранилище образов. Самый известный — публичный Docker Hub [6, стр. 23].
- REST API — интерфейс взаимодействия между CLI и демоном.

Docker-образ (image) — неизменяемый шаблон только для чтения, содержащий всё необходимое для работы приложения: обрезанную ОС, интерпретатор, библиотеки, код. Образ состоит из слоёв (layers) — каждый шаг сборки (установка пакета, копирование файлов) создаёт новый слой. Слои кешируются и переиспользуются, что экономит дисковое пространство и ускоряет сборку [1, стр. 23].

Контейнер — запущенный экземпляр образа. Контейнер имеет собственные изолированные процессы, файловую систему, сеть и ограничения ресурсов. Контейнеры легковесны и временны: по умолчанию данные внутри контейнера исчезают при его удалении [3, стр. 23].

Dockerfile — текстовый файл с инструкциями для автоматической сборки образа. В нём указывается базовый образ, копируются файлы, устанавливаются зависимости и задаётся команда запуска [1, стр. 23].

1.3 Мультиконтейнерные приложения и Docker Compose. Сравнение с оркестраторами

Хотя разрабатываемое приложение является монолитным, в production-среде для обеспечения безопасности (HTTPS) требуется запуск нескольких контейнеров: собственно приложения, reverse-проxy для обработки HTTPS-запросов и ACME-клиента для автоматического получения сертификатов.

Управлять несколькими контейнерами вручную неудобно — здесь приходит на помощь Docker Compose [5, стр. 23].

Docker Compose — инструмент для определения и запуска мультиконтейнерных приложений. Конфигурация всех сервисов (контейнеров), их сетей и томов описывается в одном файле в формате YAML — `docker-compose.yml`. Запуск всего проекта осуществляется одной командой `docker compose up` [6, стр. 23].

Основные сценарии использования Docker Compose [1, стр. 23]:

- Локальная разработка — поднятие всей инфраструктуры проекта на ноутбуке разработчика одной командой.
- Тестирование — создание изолированного окружения для автоматических тестов.
- CI/CD — автоматическое развертывание тестовых сред в конвейерах непрерывной интеграции.
- Документирование инфраструктуры — файл Compose служит одновременно кодом и документацией.

Ограничения Docker Compose [5, стр. 23]:

- Работает только на одном сервере (не распределяет контейнеры по кластеру).
- Ограниченные возможности отказоустойчивости (нет автоматического восстановления при падении узла).
- Не предназначен для продвинутого масштабирования (горизонтальное масштабирование возможно только в пределах одного хоста).

Для решения этих задач существуют системы оркестрации: Kubernetes (промышленный стандарт, очень сложен), Docker Swarm (встроен в Docker, проще, но менее функционален), Nomad (легковесная альтернатива) [10, стр. 23]. Однако для курсовой работы, где все контейнеры запускаются на одном сервере, Docker

Compose является оптимальным выбором благодаря простоте, наглядности и достаточной функциональности.

1.4 Механизмы получения и автоматического обновления SSL/TLS-сертификатов Let's Encrypt

Обеспечение безопасности передачи данных — критический аспект любого веб-приложения, особенно при работе с медицинскими данными (результаты анализов крови). Протокол HTTP передаёт данные в открытом виде, что делает возможными атаки типа Man-in-the-Middle (MITM) — перехват и подмена данных злоумышленником.

Для защиты используется протокол TLS (Transport Layer Security) и его предшественник SSL (Secure Sockets Layer). TLS обеспечивает три ключевых свойства [7, стр. 23]:

Шифрование — данные недоступны для перехвата.

Аутентификация — клиент убеждается, что общается именно с нужным сервером.

Целостность — обнаружение модификации данных в пути.

Аутентификация сервера обеспечивается цифровыми сертификатами, которые выдаются центрами сертификации (Certificate Authority, CA) — организациями, которым доверяют браузеры и операционные системы.

Традиционно получение SSL-сертификата было платным (от 50 до нескольких сотен долларов в год) и длительным процессом. В 2015 году некоммерческая организация Let's Encrypt совершила революцию, создав бесплатный, автоматизированный и открытый центр сертификации.

Ключевая особенность Let's Encrypt — полная автоматизация через протокол ACME (Automatic Certificate Management Environment). Сертификаты выдаются сроком на 90 дней — это сделано намеренно, чтобы принудить разработчиков настраивать автоматическое обновление.

Процесс получения сертификата включает испытание (challenge) — процедуру подтверждения владения доменом. Существуют два основных типа испытаний [8, стр. 23]:

- HTTP-01 challenge — ACME-клиент размещает специальный токен в скрытой директории на веб-сервере (/.well-known/acme-challenge/), а Let's Encrypt проверяет его доступность по HTTP. Требуется открытого 80-го порта и публичной доступности домена.
- DNS-01 challenge — ACME-клиент создаёт TXT-запись в DNS-зоне домена через API провайдера. Не требует открытых портов, подходит для wildcard-сертификатов и внутренних серверов.

Для работы с Let's Encrypt в Docker-среде существует несколько инструментов:

- Certbot — официальный клиент от EFF, требует ручной настройки интеграции с Docker.
- Traefik — reverse-проxy со встроенной поддержкой ACME, сложен в настройке.
- Caddy — веб-сервер с автоматическим HTTPS, менее гибок.
- nginx-проxy + acme-companion — связка из двух контейнеров, настраивается через Docker-лейблы.

Обоснование выбора: для данной работы выбрана связка nginx-проxy + acme-companion, так как она обеспечивает минимальную конфигурацию (достаточно добавить три лейбла к контейнеру приложения), автоматическое обновление сертификатов и полную прозрачность для самого приложения (Flask продолжает работать по HTTP внутри сети Docker).

Важные нюансы Let's Encrypt [7, стр. 23]:

- Сертификаты выдаются только на зарегистрированные доменные имена, не на IP-адреса или localhost.
- Короткий срок действия (90 дней) требует обязательной настройки автообновления.

- Существуют лимиты на количество сертификатов на один домен (достаточные для учебных проектов).

1.5 Обоснование выбора технологического стека и анализ альтернатив

При выборе технологий для реализации любого проекта важно понимать, какие альтернативы существуют на рынке и почему принято то или иное решение. В данном разделе рассмотрены альтернативы для каждого ключевого компонента разрабатываемой системы: среды контейнеризации, инструмента оркестрации и способа получения SSL-сертификатов.

Альтернативы Docker.

Технология контейнеризации стандартизирована организацией OSI (Open Container Initiative), что обеспечивает совместимость между различными инструментами [1, стр. 23]. Наряду с Docker существуют другие реализации, представленные в таблице 1.

Таблица 1. Сравнение технологий контейнеризации

Технология	Особенности	Сценарий использования
Podman	Не требует фонового демона, может работать без прав root, совместим с командами Docker	Высокозащищенные среды, экосистема Red Hat
containerd	Промышленный рантайм, минималистичный, используется внутри Docker	В качестве компонента Kubernetes
LXC/LXD	Запускает полноценные ОС, а не отдельные процессы	Замена виртуальным машинам
Docker	Экосистема, простота использования, лучшее сообщество	Выбран — стандарт для разработки и CI/CD

Для данной работы выбран Docker, поскольку он наиболее распространен в учебных проектах, имеет огромное количество готовых

образов и документации, а также идеально интегрируется с остальными выбранными инструментами.

Альтернативы Docker Compose.

Для управления несколькими контейнерами на одном сервере можно использовать разные подходы [5, стр. 23]:

- Podman-compose — аналог Compose для экосистемы Podman, читает те же файлы конфигурации. Не рассматривается, так как основным инструментом выбран Docker.
- HashiCorp Nomad — более мощный оркестратор, способный работать как на одном сервере, так и в кластере. Проще Kubernetes, но сложнее Compose. Избыточен для проекта с тремя контейнерами.
- Bash-скрипты с командами `docker run` — императивный подход, при котором каждый контейнер запускается отдельной командой. Такой подход приводит к трудно поддерживаемому коду, отсутствию единого управления и проблемам с обработкой ошибок.
- Docker Compose — выбран. Позволяет декларативно описать всю инфраструктуру в YAML-файле, запускать проект одной командой, управлять зависимостями и сетями.

Альтернативы Let's Encrypt и способам получения сертификатов.

Для защиты передаваемых данных необходимо использовать HTTPS. Источники сертификатов могут быть разными. Сравнение центров сертификации отражено в таблице 2 [7, стр. 23].

Таблица 2. Сравнение центров сертификации

Центр сертификации	Стоимость	Автоматизация	Срок	Характеристики
Let's Encrypt	-	Полная (ACME)	90 дней	Требуется настройка автообновления, доверен всеми браузерами. Выбран

Comodo / Sectigo	\$50–200/год	Частичная	1–2 года	Требуют ручного подтверждения, используются корпорациями
Самоподписанные	-	Ручная	Любой	Не доверяются браузерами, подходят только для разработки
Cloudflare	Бесплатно (базовый)	Полная (на стороне Cloudflare)	90 дней	Сертификат хранится у провайдера, трафик идет через его сеть

Для автоматического получения и обновления сертификатов Let's Encrypt в Docker-среде существует несколько инструментов. Сравнение ACME-инструментов для Docker описано в таблице 3 [8, стр. 23].

Таблица 3. Сравнение ACME-инструментов для Docker

Инструмент	Способ работы	Преимущества	Недостатки
Certbot	Отдельный клиент, запускаемый вручную или по расписанию	Официальный, надежный	Сложная интеграция с Docker, требует ручного управления томами
Traefik	Reverse-proxy со встроенным ACME	Полностью автоматический, интеллектуальный	Сложная конфигурация, тяжеловесный
Caddy	Веб-сервер с HTTPS по умолчанию	Очень простой	Меньше гибкости, хуже интеграция с Docker Compose
nginx-proxy + acme-companion	Два контейнера (прокси + менеджер)	Простая настройка через лейблы, автоматическое обновление	Выбран

Таким образом, технологический стек **Docker + Docker Compose + Let's Encrypt (nginx-proxy/acme-companion)** обеспечивает необходимое качество решения при минимальных временных и финансовых затратах, что полностью соответствует целям курсовой работы.

ГЛАВА 2. Практическая реализация контейнеризации и автоматизации HTTPS

2.1. Структура проекта

Перед началом контейнеризации необходимо определить структуру проекта. В корневой директории размещаются следующие файлы:

```
blood_converter/
├── app.py                # Flask-приложение (логика пересчета)
├── requirements.txt      # Зависимости Python (Flask, gunicorn)
├── Dockerfile           # Инструкция по сборке образа
├── docker-compose.yml   # Оркестрация всех сервисов
├── .env                 # Переменные окружения (домен, email)
└── templates/
    └── index.html        # HTML-шаблон пользовательского интерфейса
```

Такая структура позволяет разделить код приложения, конфигурацию контейнеризации и переменные окружения, что соответствует лучшим практикам DevOps [1, стр. 23].

2.2. Разработка Dockerfile для упаковки Flask-приложения

Dockerfile — это текстовый файл, содержащий последовательность инструкций для автоматической сборки Docker-образа. Каждая инструкция создаёт новый слой образа, что позволяет кешировать результаты и ускорять повторные сборки

```
1. FROM python:3.11-slim
2.
3. WORKDIR /app
4.
5. COPY requirements.txt .
6. COPY app.py .
7. COPY templates ./templates
```



```

8.
9. RUN pip install --no-cache-dir -r requirements.txt
10.
11. EXPOSE 5000
12.
13. CMD ["python", "app.py"]
14.

```

Инструкция	Назначение	Примечание
FROM	Определяет базовый образ, на основе которого строится новый образ	python:3.11-slim содержит Python 3.11 и минимальную ОС
WORKDIR	Создаёт и переключает на рабочую директорию внутри контейнера	Все последующие команды будут выполняться из /app
COPY	Копирует файлы с хост-системы внутрь контейнера	Первое COPY копирует только requirements.txt для кеширования слоя с зависимостями
RUN	Выполняет команду во время сборки образа	pip install устанавливает зависимости; --no-cache-dir уменьшает размер образа
EXPOSE	Документирует, какой порт приложение слушает внутри контейнера	Информационный оператор, не публикует порт на хосте
CMD	Задаёт команду, выполняемую при запуске контейнера	gunicorn — production-сервер для WSGI-приложений

Выбор Gunicorn вместо встроенного сервера Flask. Встроенный сервер Flask (`app.run()`) предназначен только для разработки. Он однопоточный, не справляется с несколькими запросами одновременно и небезопасен для production. Gunicorn — это WSGI-сервер, который создаёт несколько рабочих процессов и корректно обрабатывает параллельные запросы.

Сборка образа. Для сборки образа выполняется команда:

```
docker build -t blood-converter .
```

Где `-t blood-converter` задаёт имя образа, а `.` указывает на текущую директорию как контекст сборки.

2.3. Конфигурация Docker Compose с поддержкой HTTPS

Для развертывания приложения с поддержкой HTTPS необходимо запустить три сервиса: само Flask-приложение, reverse-proxy (nginx-proxy) для обработки HTTPS-запросов и менеджер сертификатов (acme-companion) для автоматического получения и обновления SSL-сертификатов Let's Encrypt.

Листинг docker-compose.yml представлен в приложении [Приложение А].

Пояснение сервисов представлено в таблице 4.

Таблица 4. Пояснение сервисов

Сервис	Назначение
app	Flask-приложение, которое выполняет логику пересчета лабораторных показателей
nginx-proxy	Reverse-proxy на основе nginx, принимает внешние HTTP/HTTPS-запросы и перенаправляет их контейнеру app
acme-companion	Менеджер сертификатов, взаимодействует с Let's Encrypt по протоколу ACME для получения и обновления сертификатов

Пояснение ключевых параметров представлено в таблице 5.

Таблица 5. Пояснение ключевых параметров

Параметр	Назначение
expose	Открывает порт внутри сети Docker, но не публикует его на хосте. Это безопаснее, чем ports, для внутренних сервисов
labels	Метаданные, которые считывают nginx-proxy и acme-companion. VIRTUAL_HOST указывает домен для проксирования, LETSENCRYPT_HOST — домен для сертификата
ports	Пробрасывает порты контейнера на хост-систему. Порт 80 используется для HTTP (верификация домена), порт 443 — для HTTPS (защищённый трафик)
volumes	Монтирует папки хоста или именованные тома в контейнер. /var/run/docker.sock — сокет Docker API, позволяющий

	контейнеру управлять другими контейнерами
<code>depends_on</code>	Указывает, что <code>асме-компаньон</code> должен запускаться после <code>nginx-проху</code>
<code>restart: unless-stopped</code>	Политика перезапуска: контейнер автоматически запускается после перезагрузки системы, если не был остановлен вручную

Для безопасного хранения конфиденциальных данных создаётся файл `.env`. Docker Compose автоматически подставляет эти переменные в файл конфигурации.

2.4. Принцип работы `nginx-проху` и `асме-компаньон`

Принцип работы `nginx-проху`. `nginx-проху` подключается к сокету Docker (`/var/run/docker.sock`) и отслеживает запуск новых контейнеров. При появлении контейнера с лейблом `VIRTUAL_HOST`, он автоматически создаёт конфигурацию виртуального хоста и начинает проксировать трафик с внешнего порта 80 на внутренний порт, указанный в `VIRTUAL_PORT`.

Принцип работы `асме-компаньон`. `асме-компаньон` также следит за контейнерами через сокет Docker. При обнаружении контейнера с лейблами `LETSencrypt_HOST` и `LETSencrypt_EMAIL` запускается процесс получения сертификата:

`асме-компаньон` запрашивает сертификат у Let's Encrypt через протокол ACME.

Let's Encrypt проверяет владение доменом через HTTP-01 challenge, обращаясь по адресу `http://домен/.well-known/acme-challenge/`.

После успешной проверки сертификат сохраняется в том `certs`.

`nginx-проху` перезагружает конфигурацию и начинает использовать HTTPS для указанного домена.

Автоматическое обновление сертификатов. Сертификаты Let's Encrypt действуют 90 дней. `асме-компаньон` встроен механизм автоматической

проверки срока действия сертификатов каждый день. Если до истечения сертификата осталось менее 30 дней, он автоматически обновляется без участия администратора.

2.5. Запуск приложения на сервере и получение SSL-сертификата

Для разворачивания приложения на реальном сервере с доменным именем необходимо выполнить следующие шаги:

Шаг 1. Подготовка сервера. На сервере с Ubuntu 22.04 устанавливаются Docker и Docker Compose:

```
sudo apt update sudo apt install ca-certificates curl sudo install -m 0755 -d /etc/apt/keyrings sudo
curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Шаг 2. Загрузка проекта. Файлы проекта загружаются на сервер (через scp).

Шаг 3. Настройка DNS. Для домена создаётся А-запись, указывающая на IP-адрес сервера. Это необходимо, чтобы Let's Encrypt мог проверить владение доменом.

Шаг 4. Запуск. Выполняется команда:

```
docker compose up -d
```

Шаг 5. Проверка. После запуска (обычно 30–60 секунд) сертификат автоматически получается и применяется. Приложение становится доступно

по адресу <https://домен>. Зелёный замок в адресной строке браузера подтверждает валидность сертификата Let's Encrypt (рисунок 1).



Рисунок 1. Сертификат Let's Encrypt

Управление сервисом после запуска представлено в таблице:

Таблица 6. Управление сервисом после запуска

Действие	Команда
Запуск	docker compose up -d
Остановка	docker compose down
Перезапуск	docker compose restart
Просмотр статуса	docker compose ps
Просмотр логов	docker compose logs -f
Обновление кода	docker compose build app && docker compose up -d

ЗАКЛЮЧЕНИЕ

В ходе практической части курсовой работы были достигнуты следующие результаты:

- Разработано веб-приложение на Flask для пересчета лабораторных единиц анализа крови
- Разработан Dockerfile для упаковки Flask-приложения в Docker-образ.
- Создан docker-compose.yml с тремя сервисами: Flask-приложение, nginx-проxy и acme-companion.
- Настроена автоматизация выпуска сертификатов Let's Encrypt через acme-companion.
- Составлена документация по запуску, включающая подготовку сервера, настройку DNS и выполнение команд Docker Compose.

Таким образом, разработанное решение полностью соответствует поставленным задачам. Ознакомиться с запущенным веб-приложением можно в приложении [Приложение Б].

QR-код на репозиторий в github с документацией по проекту представлен на рисунке 3.



Рисунок 3. GitHub репозиторий

ЛИТЕРАТУРА

1. Гринберг, Д. Docker для разработчиков и администраторов / Д. Гринберг. — СПб.: БХВ-Петербург, 2021. — 352 с.
2. Гринберг, М. Flask. Создание веб-приложений на Python / М. Гринберг. — М.: ДМК Пресс, 2020. — 280 с.
3. Рубальский, Д. Г. Контейнеризация веб-приложений на платформе Docker / Д. Г. Рубальский // Научно-технический вестник информационных технологий, механики и оптики. — 2024. — № 2. — С. 67–72.
4. Николаев, В. В. Сравнительный анализ инструментов оркестрации контейнеров / В. В. Николаев // Информационные технологии и вычислительные системы. — 2023. — № 4. — С. 112–119.
5. Рейтц, К. Python и DevOps. Автоматизация развертывания приложений / К. Рейтц. — М.: Вильямс, 2022. — 416 с.
6. Официальная документация Docker [Электронный ресурс]. — Режим доступа: <https://docs.docker.com/> (дата обращения: 10.06.2026).
7. Официальная документация Let's Encrypt [Электронный ресурс]. — Режим доступа: <https://letsencrypt.org/ru/docs/> (дата обращения: 10.06.2026).
8. Документация nginx-проху / acme-companion [Электронный ресурс]. — Режим доступа: <https://github.com/nginx-proxy/nginx-proxy> (дата обращения: 10.06.2026).
9. Официальная документация Flask [Электронный ресурс]. — Режим доступа: <https://flask.palletsprojects.com/> (дата обращения: 10.06.2026).
10. Мэтьюз, А. Kubernetes: руководство для начинающих / А. Мэтьюз. — М.: ДМК Пресс, 2021. — 320 с.

11.Контейнеризация // Википедия. —

URL: <https://ru.wikipedia.org/wiki/Контейнеризация> (дата обращения: 10.06.2026).

ПРИЛОЖЕНИЕ А

```
1. services:
2.   nginx-proxy:
3.     image: jwilder/nginx-proxy
4.     ports:
5.       - "80:80"
6.       - "443:443"
7.     volumes:
8.       - conf:/etc/nginx/conf.d
9.       - vhost:/etc/nginx/vhost.d
10.      - html:/usr/share/nginx/html
11.      - dhparam:/etc/nginx/dhparam
12.      - certs:/etc/nginx/certs:ro
13.      - /var/run/docker.sock:/tmp/docker.sock:ro
14.     restart: unless-stopped
15.
16. letsencrypt:
17.   image: jrcs/letsencrypt-nginx-proxy-companion
18.   container_name: nginx-proxy-le
19.   depends_on:
20.     - nginx-proxy
21.   volumes:
22.     - conf:/etc/nginx/conf.d
23.     - html:/usr/share/nginx/html
24.     - dhparam:/etc/nginx/dhparam
25.     - certs:/etc/nginx/certs:ro
26.     - /var/run/docker.sock:/tmp/docker.sock:ro
27.   restart: unless-stopped
28.
29. blood-converter:
30.   build: .
31.   container_name: blood_converter
32.   expose:
33.     - "5000"
34.   environment:
35.     - FLASK_ENV=production
36.     - VIRTUAL_HOST=converter.cikkikik.ru
37.     - LETSENCRYPT_HOST=converter.cikkikik.ru
38.     - LETSENCRYPT_EMAIL=arina.chikina05@gmail.com
39.   restart: unless-stopped
40.
41. volumes:
42.   conf:
43.   vhost:
44.   html:
45.   dhparam:
46.   certs:
47.
```

ПРИЛОЖЕНИЕ Б

The screenshot shows a web browser window with the address bar displaying 'converter.clickik.ru'. The page title is 'Пересчет лабораторных единиц крови и витаминов'. The main heading is 'Пересчет лабораторных единиц' with a subtitle 'Конвертация показателей крови и витаминов между разными системами измерения'. The form includes a dropdown menu for 'Выберите показатель' with 'Глюкоза' selected, an input field for 'Введите значение' with '95.5', and a 'Перевести' button. The result section shows 'Результат пересчета' as '5.3 ммоль/л' and includes the formula 'Формула: ммоль/л = мг/дл × 0.0555' and 'Исходное значение: 95.5 мг/дл'.

converter.clickik.ru Пересчет лабораторных единиц крови и витаминов

Пересчет лабораторных единиц

Конвертация показателей крови и витаминов между разными системами измерения

Выберите показатель

Глюкоза

Введите значение

95.5

Перевести

Результат пересчета

5.3 ммоль/л

Формула: ммоль/л = мг/дл × 0.0555
Исходное значение: 95.5 мг/дл

Рисунок 2. Запущенное на сервере приложение